

SCHEMA VALIDATORS

Here, in the file these are the salient features:

- **Fail soft, not silent.** Bad data is isolated, logged, and kept, never ignored and never allowed to crash the stream.
- **Result objects over `None` or exceptions.** The caller gets structured information it can count and act on.
- **Generics (`T`).** One function works for many schemas without losing type safety.
- **Separation of concerns.** Schema validation, domain validation, and storage are separate stages.

Initially, the main question we must answer in the data sent over here is, Is the payload shaped correctly? Payload is the raw data that a source sends you, for a river gauge that can be:

```
{"sensor_id": "g1", "lat": 28.2, "lon": 83.9, "value": 2.1}
```

And over here the shape is the structure of the raw data.

Similarly, this also checks if the values satisfy the rules in the `TelemetryData`. So, it never crashes the pipeline on the bad input but gives the result we can expect. Here, I have divided the file in six

sectors while I will discuss individually here.

1. Logger and Type variable

`log` replaces `print` so messages carry a name, a level, and can be routed to files or monitoring tools. `T` is a placeholder meaning "any pydantic model". It lets the same function validate `TelemetryData` today and other schemas later, with correct type hints.

```
log =  
logging.getLogger("prahari.ingestion.sche  
ma_validator")  
T = TypeVar("T", bound=BaseModel)
```

2. Field Error

This describes one failed field: where it failed (`lat`), why (`Input should be less than or equal to 90`), and the error category. It deliberately has no field for the bad value, so logs and dead-letter records can't leak sensitive data.

`frozen=True` makes it immutable, so nothing can alter it after creation.

```
@dataclass(frozen=True)  
class FieldError:  
    path: str  
    message: str  
    type: str
```

3. Validation Outcome

This is the result object, and it replaces your original `return None`. Success means `data` holds the validated model and `errors` are empty. Failure means `data` is `None` and `errors` explain why. The caller checks `outcome.ok` and never has to guess.

```

@dataclass(frozen=True)
class
ValidationOutcome(Generic[T]):
    data: Optional[T] = None
    errors: list[FieldError] =
    field(default_factory=list)
    @property
    def ok(self) -> bool: return
    self.data is not None

```

4.Reject

This packages a failed payload with its origin and reasons, ready to write to a dead-letter store. You keep bad data so you can debug it or replay it after fixing a schema.

```

@dataclass(frozen=True)
class Reject:
    source: str
    raw: Any
    errors: list[FieldError]

```

5.Validation Payload

STAGE I:

Your original

`TelemetryData(**raw_payload)` would raise a `TypeError` on `None` or a list. Here that case is caught first and turned into a clean rejection.

```

if not isinstance(raw_payload, dict):
    ...
    return
ValidationOutcome(errors=[err])

```

STAGE II:

`model_validate` is pydantic v2's recommended way to build a model from a dict. On failure, pydantic raises `ValidationError` containing every problem at once, not just the first. The list comprehension converts each into a `FieldError`, and `include_input=False` strips out the offending values. A path like

`["location", "lat"]` is joined into `"location.lat"` for readability. Then it logs a warning with the source name and the first five errors, so a badly broken payload can't flood your logs.

```

try:
    return
ValidationOutcome(data=model m
odel_validate(raw_payload))
except ValidationError as exc:
    errors = [FieldError(...) for e
in
exc.errors(include_url=False
e, include_input=False)

```

6.Validation Batch

This loops over many payloads and splits them into two lists. `valid` holds the models that passed. `rejects` holds `Reject` objects for the rest. A connector's `parse()` output can go through this in one call, and the valid list continues down the pipeline while the rejects go to the dead-letter store.

```

valid, rejects =
validate_batch(payloads,
source="river_gauge")

```

HOW CONNECTED?

connector.fetch() → connector.parse() →
 schema_validator → domain validators →
 pipeline

└── rejects
 → dead-letter store

This stage is the shape check only.
Whether a river level of 40 meters is
physically plausible is a later stage's job,
because that depends on the station, not
the schema.